

S.O.U.N.D. SYSTEM-IKER

MASTERANWEISUNG FÜR CHATGPT WORK

Vom vorhandenen Projektstand bis zum vollständig implementierten, releasefähigen Prototypen -
Version 1.1

BONUSFEATURE: BUNKER BLACKBOX - deterministischer Szenario-Recorder, Replay, Zustandsdiff, Golden Scenarios und datensparsame Bug Capsule.

Diese Datei ist als verbindlicher Copy-and-Paste-Arbeitsvertrag für ChatGPT Work formuliert. Der Auftrag endet nicht mit Planung oder Teilerfolg, sondern erst nach finalem Clean-Room-Lauf, vollständigen Release-Gates, Entwicklerdokumentation und dreistufiger Laienanleitung.

S.O.U.N.D. SYSTEM-IKER - MASTERANWEISUNG FÜR CHATGPT WORK

Version 1.1 - verbindlicher End-to-End-Arbeitsauftrag mit BUNKER BLACKBOX

ZWECK DIESER ANWEISUNG

Diese Anweisung ist als vollständiger Arbeitsauftrag für ChatGPT Work gedacht. Sie soll nicht nur analysieren oder planen, sondern das bestehende Projekt S.O.U.N.D. SYSTEM-IKER vom aktuell vorhandenen Stand bis zu einem vollständig implementierten, nachvollziehbar geprüften und releasefähigen Prototypen führen.

Der Auftrag endet ausdrücklich NICHT mit:

- einer Architekturübersicht,
- einem Pflichtenheft,
- einer Liste offener Aufgaben,
- Mockups ohne Funktion,
- Pseudocode,
- Codefragmenten,
- Platzhaltern,
- nur teilweise implementierten Kernsystemen,
- einer bloßen Testempfehlung,
- einer nur beschriebenen Release-Checkliste,
- oder dem Hinweis, was ein Entwickler später noch tun müsse.

Der Auftrag ist erst abgeschlossen, wenn der letzte verpflichtende Entwicklungs- und Prüfschritt tatsächlich ausgeführt, das Ergebnis in einem frischen Prüfkontext erneut validiert und die Abschlussdokumentation erstellt wurde.

WICHTIGE PROFESSIONELLE PRÄZISIERUNG

"Perfekt" und "fehlerfrei" sind keine seriös beweisbaren Softwareeigenschaften. Das verbindliche Ziel lautet deshalb:

"Releasefähig nach klaren, dokumentierten, reproduzierbaren Qualitäts-, Robustheits-, Sicherheits-, Wartbarkeits-, Bedienbarkeits- und Regression-Gates, ohne bekannte releaseblockierende Fehler."

Alle nicht blockierenden bekannten Einschränkungen müssen ausdrücklich dokumentiert werden. Ein Release darf niemals nur deshalb als erfolgreich gelten, weil die Anwendung startet.

NEU IN VERSION 1.1 - PROFESSIONELLES BONUSFEATURE

Diese Version integriert als verbindliches Bonusfeature die BUNKER BLACKBOX:

einen deterministischen Szenario-Recorder mit Replay-, Diff-, Diagnose- und Bug-Capsule-Funktion.

Die BUNKER BLACKBOX ist kein dekoratives Debug-Menü. Sie ist Teil der Qualitätsarchitektur und dient gleichzeitig:

- reproduzierbarer Fehlersuche,
- Regressionserkennung,
- Langzeitwartung,
- verständlicher Supportdiagnose,
- reproduzierbaren Demonstrationen,
- und dem finalen Clean-Room-Nachweis.

Sie zeichnet keine beliebigen Bildschirmbewegungen auf, sondern fachlich relevante Commands, Seeds, Versionsstände, Content Locks, erwartete Checkpoints und daraus entstandene Events. Dadurch bleiben Replays stabiler als rohe Maus-/Pixelaufzeichnungen.

Lokale MP3-Dateien, Nutzerbilder, Secrets und unnötige persönliche Inhalte werden niemals automatisch Bestandteil einer Bug Capsule.

0. VERBINDLICHER AUFTRAG, QUELLENHIERARCHIE UND NICHT VERHANDELBARE REGELN

0.1 AUFTRAG

Bearbeite das Projekt selbstständig, systematisch und iterativ bis zum vollständig releasefähigen Prototypen. Nutze vorhandene Dateien, bestehende Spezifikationen, frühere Entwicklungsstände und Projektartefakte als Ausgangspunkt. Führe tatsächliche Änderungen durch. Prüfe jede Änderung. Dokumentiere die Evidenz.

0.2 QUELLENHIERARCHIE

Bei Widersprüchen gilt folgende Reihenfolge:

- 1. Diese Masteranweisung v1.1.**
- 2. Die aktuell eingefrorene Master-Spezifikation des Projekts, insbesondere MASTER-Entwicklungsplan v0.4 oder eine explizit neuere freigegebene Version.**
- 3. Der Master-Ausführungsprompt v1.0 oder eine explizit neuere freigegebene Version.**
- 4. Der eingefrorene I00-Baseline-Stand und seine Architekturverträge.**
- 5. Spätere, ausdrücklich freigegebene ADRs, Change Records und Migrationsentscheidungen.**
- 6. Ältere Brainstorms, Entwürfe und Varianten.**
- 7. Implizite Annahmen.**

Wenn eine ältere Quelle mit einer neueren verbindlichen Quelle kollidiert, darf die ältere Quelle nicht stillschweigend gewinnen.

0.3 KEINE TECHNISCHEN PLATZHALTER

Im Releasezustand sind verboten:

- TODO, FIXME, HACK, XXX als offene Entwicklungsmarker,
- Ellipsen als ausgelassener Programmcode,
- Funktionen, die nur "not implemented" werfen,
- Dummy-Rückgaben für verpflichtende Kernlogik,
- leere Eventhandler, die nur zukünftige Funktionen vortäuschen,
- fest codierte Beispielwerte an Stellen, an denen echte Daten erwartet werden,
- UI-Elemente ohne Funktion, wenn sie wie aktive Funktionen dargestellt werden,
- schematische JSON-Beispiele anstelle der tatsächlich verwendeten Produktionsschemas,
- simulierte Prüfergebnisse ohne tatsächliche Prüfausführung.

Bewusst unbestätigte Storyinhalte dürfen definierte Zustände wie UNSET, DRAFT oder VARIANT besitzen, sofern das Schema diese Zustände ausdrücklich erlaubt. Das ist kein technischer Platzhalter.

0.4 KEIN BLINDES WEITERBAUEN

Vor jeder wesentlichen Änderung:

- aktuelle Struktur erfassen,
- Abhängigkeiten bestimmen,
- betroffene Module identifizieren,
- erlaubte Änderungsscope definieren,
- Regression-Risiken bestimmen,
- Backup/Snapshot oder reproduzierbaren Rückweg sicherstellen.

0.5 KEIN STOPP NACH PLANUNG

Planung ist nur der erste Teil des Auftrags. Nach der Planung muss die Umsetzung ohne erneute Nutzerfreigabe fortgesetzt werden, sofern kein echter externer Blocker besteht. Bei kleineren Unklarheiten ist die robusteste, wartbarste und mit der Master-Spezifikation konsistente Variante selbstständig zu wählen und als ADR/Change Record zu dokumentieren.

0.6 BEST-EFFORT STATT RÜCKFRAGEN

Rückfragen nur dann, wenn:

- eine externe Zugangsdatenentscheidung zwingend erforderlich ist,
- eine irreversible reale Aktion außerhalb des Projekts betroffen wäre,
- mehrere Alternativen rechtlich oder sicherheitstechnisch wesentlich unterschiedlich wären,
- oder die vorhandenen Quellen objektiv keine tragfähige Entscheidung zulassen.

Ansonsten selbst entscheiden, dokumentieren und weiterarbeiten.

0.7 REPRODUZIERBARKEIT

Jeder grüne Status muss durch einen anderen Prüflauf reproduzierbar sein. "Hat einmal funktioniert" ist kein Gate.

0.8 OFFLINE-FIRST ALS GRUNDPRINZIP

Der Prototyp muss ohne permanente Internetverbindung spielbar sein, soweit die definierte Kernfunktion dies zulässt. Synchronisation darf als separater Prozess stattfinden. Ein Netzausfall darf nicht zum Verlust lokaler gültiger Spieldaten führen.

0.9 DATEN VOR DARSTELLUNG

Die UI darf persistente Zustände nicht direkt manipulieren. Fachliche Änderungen laufen über definierte Commands/Services/Event-Pipelines. Projektionen und UI lesen daraus abgeleitete Zustände.

0.10 KLEINE ÄNDERUNGEN BLEIBEN KLEIN

Ein Austausch eines Introtexts, Gerüchts, Graffiti-Hinweises oder NPC-Satzes darf keinen Eingriff in den Game-Core erfordern. Content muss versioniert, validiert, registriert und austauschbar sein.

1. GLOBALE PROJEKTSTANDARDS

1.1 TECHNOLOGISCHE BASIS

Zielplattform des Prototyps:

- Browser-first,
- HTML5,
- CSS,
- modernes JavaScript als ES Modules,
- IndexedDB für persistente lokale Daten,
- Service Worker/PWA nur dort, wo stabil und sinnvoll,
- Web Audio API für lokale Audiofunktionen,
- Web Crypto API für Hash-/Signaturfunktionen,
- keine unnötige Framework-Abhängigkeit.

Eine bereits etablierte, begründete und getestete Abweichung darf beibehalten werden, wenn sie im Architekturvertrag dokumentiert ist.

1.2 MODULARITÄT

Mindestens folgende logische Schichten strikt trennen:

- UI/Presentation,
- Application/Use Cases,
- Domain/Game Core,
- Event/Ledger,
- Persistence,
- Sync/Conflict Resolution,
- Content/Story,
- Audio,
- Validation,
- Diagnostics/Recovery,
- Tests,
- Tooling,
- Documentation.

Zyklische Modulabhängigkeiten sind zu vermeiden und durch automatisierte Architekturtests zu verhindern.

1.3 STATE OWNERSHIP

Jeder persistente Zustand hat genau einen fachlichen Owner. Andere Module dürfen ihn nur über definierte Schnittstellen beeinflussen. Erstelle und pflege eine maschinenlesbare State-Owner-Matrix für mindestens:

- Character,
- Skills,
- Relationships,
- Housing,
- Rooms,
- Graffiti,
- Council/Voting,
- Economy,
- Rumors,
- Events/Parties,
- Biography,
- World,
- Content Registry,
- Sync Metadata,
- Audio Metadata.

1.4 IDs

Alle dauerhaft referenzierten Entitäten besitzen stabile, namespacetechnisch eindeutige IDs. IDs dürfen nicht von Anzeigenamen abhängen. Umbenennen darf keine historische Referenz brechen.

1.5 VERSIONIERUNG

Verwende SemVer-kompatible Versionierung für Softwarepakete und definierte Schema-/Content-Versionen. Content, Schema, Ruleset, World Format und Core dürfen getrennt versioniert werden.

1.6 CHANGE MANAGEMENT

Jede nicht triviale Änderung erhält:

- Change-ID,
- Ziel,
- Scope,
- betroffene Dateien/Module,
- Risiko,
- Testplan,
- Ergebnis,
- Regression-Evidence,
- Rollbackhinweis,
- finalen Status.

1.7 ARCHITECTURE DECISION RECORDS

Architekturentscheidungen werden als ADR mit Kontext, Entscheidung, Alternativen, Folgen und Status dokumentiert. Revidierte ADRs werden nicht gelöscht, sondern ersetzt/deprecated und verlinkt.

1.8 CODING STANDARD

Einheitlich:

- klare, sprechende englische technische Bezeichner,
- UI-/Storytexte in Content-Dateien,
- kleine Funktionen mit klarer Verantwortung,
- keine versteckten globalen Mutable States,
- keine magischen Zahlen ohne benannte Konstante oder Regeldefinition,
- keine stillen Fehler,
- keine unstrukturierten "catch-all"-Fehlerbehandlungen.

1.9 FORMATIERUNG UND LINTING

Ein automatisierter Format-/Lint-Pfad muss existieren. Änderungen dürfen nur grün eingefroren werden, wenn Format, Lint und Syntaxprüfung bestanden sind.

1.10 SCHEMA-STANDARD

Strukturierte Daten mit JSON Schema Draft 2020-12 oder einer explizit dokumentierten kompatiblen Variante validieren. Schemas selbst versionieren und gegen Positiv- und Negativfixtures testen.

1.11 CONTENT-STANDARD

Content-Pakete benötigen mindestens:

- contentId,
- version,
- schemaVersion,
- locale,
- type,
- status,

- compatibility,
- dependencies,
- checksum,
- provenance,
- optional supersedes/deprecation metadata.

Aktivierung nur nach erfolgreicher Validierung.

1.12 CONTENT-KANON

Charakter-/Storydaten unterscheiden:

- CANON,
- DRAFT,
- VARIANT,
- RETIRED,
- UNSET.

Nur CANON darf ohne explizite Storyregel als verbindliche Weltwahrheit behandelt werden.

1.13 ERROR CODES

Definiere stabile Fehlercodes statt nur Freitextmeldungen. Fehlercode, technische Ursache, nutzerfreundliche Meldung, Schweregrad, Recoverability und empfohlene Aktion müssen getrennt abbildbar sein.

1.14 LOGGING

Logs:

- datensparsam,
- strukturiert,
- reproduzierbar,
- ohne unnötige persönliche Inhalte,
- mit Correlation-ID,
- mit Severity,
- mit Modul,
- mit Zeitbezug,
- mit sicherem Export für Support/Analyse.

1.15 DATENSCHUTZ

Lokale MP3-Dateien und lokale Nutzerbilder werden nicht automatisch synchronisiert oder hochgeladen. Metadaten und Freigaben werden getrennt behandelt. Kein stilles Senden lokaler Mediendateien.

1.16 ACCESSIBILITY

WCAG-2.2-orientierte Bedienbarkeit:

- Tastaturbedienung,
- sichtbarer Fokus,
- semantische Struktur,
- ausreichende Kontraste,
- Reduced Motion,
- keine essenzielle Information ausschließlich über Farbe,
- 200-%-Zoom ohne Funktionsverlust,
- Flash-/Strobe-sichere Standardmodi.

1.17 RESPONSIVE UI

Desktop ist primärer Entwicklungsfokus, aber Kernfunktionen dürfen bei kleineren Ansichten nicht unbenutzbar werden. Breakpoints und minimale Interaktionsgrößen dokumentieren.

1.18 PERFORMANCE

Definiere messbare Budgets für:

- Initial Load,
- Speicherverbrauch,
- IndexedDB-Operationen,
- Projektions-Neuberechnung,
- Event-Replay,
- große Biografie,
- große Graffiti-Historie,
- Audio-Visualisierung.

Budgets müssen durch Tests oder Messskripte geprüft werden.

1.19 SECURITY

Mindestens:

- strikte Inputvalidierung,
- keine Ausführung importierter Inhalte als Code,
- CSP-konforme Architektur soweit praktikabel,
- sichere Dateityp-/Dateigrößenprüfung,
- keine Secrets im Clientrepository,
- Integritätsprüfung importierter Pakete,
- Schutz vor offensichtlicher DOM-Injection,
- sichere Serialisierung.

1.20 PORTABILITÄT

Der Prototyp muss auf aktuellen Desktop-Browsern unter Linux und mindestens einer zweiten Desktopplattform funktionieren. Browser-spezifische Fähigkeiten sind über Feature Detection statt User-Agent-Hardcoding zu behandeln.

1.21 REPRODUZIERBARES SZENARIOFORMAT

Fachlich relevante Test- und Fehlerszenarien müssen in einem versionierten, maschinenlesbaren Format gespeichert werden können. Ein Szenario enthält mindestens:

- scenarioId,
- scenarioVersion,
- title,
- purpose,
- coreVersion,
- rulesetVersion,
- worldFormatVersion,
- schemaVersion,
- contentLockFingerprint,
- initialWorldFixture,
- deterministicSeed,
- orderedCommands,
- expectedCheckpoints,
- expectedFinalFingerprint,
- privacyProfile.

Das Format darf keine lokalen Audio- oder Bilddateien einbetten, sofern dies nicht ausdrücklich und sicher freigegeben wurde.

2. SUBAGENTEN-/FACHROLLENMODELL

2.1 ORCHESTRATOR

Eine zentrale Rolle besitzt Merge- und Releaseverantwortung. Sie zerlegt Arbeit, verhindert Scope-Kollisionen, sammelt Evidence und entscheidet anhand der Gates.

2.2 ARCHITEKTUR-REVIEW

Prüft:

- Modulgrenzen,
- Dependency-Richtung,
- State Ownership,
- API-Verträge,
- ADR-Konformität,
- langfristige Wartbarkeit.

2.3 DOMAIN-/GAMEPLAY-REVIEW

Prüft:

- Spielregeln,
- Kausalität,
- emergente Wechselwirkungen,
- irreversible Entscheidungen,
- Fairness,
- Anti-Softlock,
- Konsistenz mit Spielphilosophie.

2.4 DATA-/SYNC-REVIEW

Prüft:

- Event-Schema,
- Persistenz,
- Replay,
- Hash-/Signaturketten,
- Offline-Konvergenz,
- Konfliktauflösung,
- Migration,
- Import/Export,
- Backup/Restore.

2.5 QA-/REGRESSION-REVIEW

Prüft:

- Testabdeckung,
- Negativfälle,
- Regression,
- End-to-End-Flows,
- Browsermatrix,
- Langlauf-tests,

- Wiederholbarkeit,
- Golden-Replays der BUNKER BLACKBOX,
- deterministische Checkpoint- und Fingerprintvergleiche.

2.6 SECURITY-/INTEGRITY-REVIEW

Prüft:

- Eingabegrenzen,
- Importpfade,
- Integrität,
- Fehlerlecks,
- DOM-/Content-Sicherheit,
- Rechte-/Capability-Grenzen,
- lokale Mediendaten.

2.7 UX-/ACCESSIBILITY-REVIEW

Prüft:

- Verständlichkeit,
- Zustandsfeedback,
- Fehlermeldungen,
- Tastatur,
- Zoom,
- Reduced Motion,
- Strobe-Sicherheit,
- Laienbedienung.

2.8 CONTENT-/CANON-REVIEW

Prüft:

- Storykonsistenz,
- Canon-Status,
- stabile IDs,
- Lokalisierung,
- Textlängen,
- subtile Hinweise,
- keine ungewollte Spoilerung.

2.9 RELEASE-/EVIDENCE-REVIEW

Prüft:

- Versionskonsistenz,
- Changelog,
- Manifest,
- Checksums,
- reproduzierbaren Build/Start,
- frischen Installationslauf,
- Dokumentationsvollständigkeit,
- Releasepaket,
- mindestens einen erfolgreichen Golden-Replay aus einem frischen Prüfzustand,
- Erzeugung und Wiederverwendung einer datensparsamen Bug Capsule.

2.10 AUSFÜHRUNG

Wenn ChatGPT Work echte getrennte Agenten/Unteraufgaben parallel verwenden kann, nutze sie mit klar abgegrenzten Scopes. Wenn nicht, führe dieselben Rollen als strikt getrennte sequenzielle Review-Pässe durch. Kennzeichne den Modus ehrlich. Qualität darf nicht davon abhängen, ob Parallelität verfügbar ist.

3. DREISTUFIGER END-TO-END-ARBEITSABLAUF

STUFE 1 - BESTANDSAUFNAHME, KONSOLIDIERUNG UND VERBINDLICHE BASELINE

ZIEL

Vor jeder weiteren Featureentwicklung muss eindeutig feststehen, was existiert, was verbindlich ist, was fehlt und wie der Projektzustand reproduzierbar geprüft wird.

3.1.1 PROJEKT FINDEN UND INVENTARISIEREN

- Alle relevanten Projektdateien lokalisieren.
- Bestehende ZIPs/Artefakte entpacken, falls notwendig.
- Repositorybaum erfassen.
- Versionen, Manifeste, Schemas, Tests, Dokumente, ADRs und Change Records erfassen.
- Doppelte oder konkurrierende Quellen identifizieren.
- Keine Datei aufgrund ihres Namens ungeprüft als aktuell annehmen.

3.1.2 BASELINE VALIDIEREN

Führe vorhandene Checks aus. Prüfe:

- Startbarkeit,
- Syntax,
- JSON-Parsebarkeit,
- Schemavalidierung,
- Unit-Tests,
- Architekturtests,
- vorhandene Regressionstests,
- Fingerprints/Checksums.

Wenn der vorhandene Stand rot ist, zuerst Baseline reparieren. Neue Features erst danach.

3.1.3 MASTERQUELLEN EINFRIEREN

Erzeuge einen Baseline-Record mit:

- Projektversion,
- Ruleset-Version,
- Schema-Version,
- Content-Version,
- World-Format-Version,
- aktuellem Fingerprint,
- Datum/Zeit,
- Prüfmodus,
- bekannter Toolchain.

3.1.4 GAP-MATRIX

Erstelle eine vollständige Matrix:

- Anforderung,
- Quelle,
- aktuelle Implementierung,
- Status: PASS/PARTIAL/MISSING/BLOCKED,
- Risiko,
- Zieliteration,
- benötigte Tests,

- Abhängigkeiten.

Kein "später" ohne konkrete Iterationszuordnung.

3.1.5 KONSISTENZPRÜFUNG

Prüfe explizit:

- Namenskonflikte,
- ID-Konflikte,
- Schemawidersprüche,
- State-Owner-Dopplungen,
- konkurrierende Regeln,
- unklare Versionsbeziehungen,
- uneindeutige Persistenzwege,
- doppelte Businesslogik,
- UI-Direktzugriffe auf persistente Daten.

3.1.6 ARCHITEKTURVERTRAG

Aktualisiere den Architecture Contract so, dass er maschinenprüfbar beschreibt:

- erlaubte Abhängigkeiten,
- verbotene Abhängigkeiten,
- öffentliche Modulgrenzen,
- State Owners,
- Contentgrenzen,
- Plugin-/Capability-Grenzen,
- Persistence-Grenzen,
- Sync-Grenzen.

3.1.7 TESTSTRATEGIE

Definiere mindestens:

- Unit,
- Schema,
- Contract,
- Property/Invariant,
- Integration,
- End-to-End,
- Migration,
- Replay,
- Sync permutations,
- Recovery,
- Accessibility smoke,
- Performance smoke,
- Security negative,
- Release smoke.

3.1.8 RELEASEDEFINITION

Der Prototyp gilt nur dann als releasefähig, wenn alle später unter STUFE 3 definierten Pflichtgates grün sind.

3.1.9 STUFE-1-ERGEBNIS

Stufe 1 endet nur mit:

- grüner oder bewusst dokumentiert reparierter Baseline,

- vollständiger Gap-Matrix,
- priorisiertem Iterationsplan,
- aktualisiertem Architekturvertrag,
- aktualisierten Schemas/Standards,
- dokumentierter Toolchain,
- eindeutigem Ausgangsfingerprint.

Danach automatisch mit Stufe 2 fortfahren.

STUFE 2 - VOLLSTÄNDIGE IMPLEMENTIERUNG DES RELEASEFÄHIGEN PROTOTYPEN

ZIEL

Alle für den Prototyp verpflichtenden Systeme werden tatsächlich implementiert, integriert, getestet und zu einem zusammenhängenden Spielablauf verbunden.

Die Reihenfolge darf technisch angepasst werden, aber kein Pflichtsystem darf ausgelassen werden.

3.2.1 APP SHELL UND STARTPFAD

Implementiere:

- zuverlässigen App-Start,
- Boot-Sequenz,
- Health Check,
- Lade-/Fehlerzustände,
- Feature Detection,
- Recovery-Einstieg,
- sichtbare Version/Build-ID im Diagnosebereich.

ABNAHME

Frischer Start ohne vorherige Daten funktioniert. Defekte Persistenz führt nicht zu weißem Bildschirm, sondern in einen sicheren Diagnose-/Recoverypfad.

3.2.2 DESIGN SYSTEM UND UI-GRUNDSYSTEM

Implementiere:

- Design Tokens,
- Typografie,
- Spacing,
- Fokuszustände,
- Buttons,
- Panels,
- Dialoge,
- Tabs,
- Statuschips,
- Toast/Notification,
- Empty States,
- Error States,
- Reduced Motion,
- kontrasttaugliche Varianten.

Keine Kernansicht darf eigene inkompatible Basisstile erfinden.

3.2.3 HAUPTANSICHT BUNKER

Implementiere die gemeinsame Hauptansicht mit:

- Bunkeraußenwand,
- Gelände/Bauflächen,
- Graffitibereich,
- Ratsplakat,
- Zugängen,
- Status- und Ereignisfeedback,

- permanenter Charakterkarte,
- sichtbaren Weltveränderungen.

Die Ansicht muss aus Spielzustand gerendert werden und nach Reload reproduzierbar identisch rekonstruierbar sein.

3.2.4 CHARAKTERSYSTEM

Implementiere:

- stabile Character IDs,
- Basisdaten,
- Skills,
- Ruf/Szenestatus,
- visuelle Abzeichen/Spuren,
- Beziehungen,
- Besitzbezüge,
- Wohnstatus,
- Bio-Verknüpfungen.

Feste Storyfiguren als Ausgangsbestand:

- pppoppi poppsen von bückstücken,
- Flash Milchmädchen Pete,
- Edna Ruppen.

Sie sind persistente Storyfiguren und nicht als normale Spieler-Slots zu behandeln.

3.2.5 CHARAKTERKARTE

Implementiere:

- markantes Avatarfeld,
- Rang/Level/Szenestatus,
- wenige gut lesbare Primärwerte,
- sichtbare Leistungs-/Geschichtsspuren,
- Bio-Button,
- Accessibility Labels.

Option für Nutzerbild nur mit lokaler Verarbeitung und sicherer Dateiprüfung. Standardcharaktere müssen ohne Upload vollständig funktionieren.

3.2.6 BIOGRAFIE

Biografie wird aus Ereignissen abgeleitet, nicht als freie unstrukturierte Parallelhistorie geführt.

Implementiere:

- chronologische Ereignisse,
- Relevanzstufen,
- Kapitelbildung,
- Quellen-/Perspektivkennzeichnung,
- Verknüpfung zu Beziehungen/Räumen/Events,
- Filter,
- reproduzierbare Neuerzeugung aus Eventbestand.

Einzelne Meinungen/Gerüchte dürfen nicht als objektive Fakten dargestellt werden.

3.2.7 EVENT-SOURCING / LEDGER

Implementiere:

- immutable Event IDs,
- Author/Device ID,

- Character/Object IDs,
- Event Type,
- Event Version,
- Ruleset Version,
- Timestamp plus logische Reihenfolge,
- Causation ID,
- Correlation ID,
- Payload,
- canonical serialization,
- hash,
- Signaturmechanismus soweit im lokalen Modell vorgesehen.

Validierte Events werden append-only behandelt. Korrekturen erfolgen durch neue Events, nicht durch heimliches Umschreiben historischer Ereignisse.

3.2.8 PROJEKTIONEN

Implementiere deterministische Projektionen für mindestens:

- Charakterzustand,
- Beziehungen,
- Geld,
- Besitz,
- Wohnstatus,
- Räume,
- Graffiti,
- Rat,
- Gerüchte,
- Veranstaltung,
- Biografie.

Replay desselben gültigen Eventsets muss denselben Zustand ergeben.

3.2.9 PERSISTENZ

IndexedDB-Schicht mit:

- versionierten Stores,
- Transaktionen,
- atomarem Schreiben,
- Fehlerbehandlung,
- Migration,
- Export,
- Import,
- Snapshot,
- Recovery.

Kein kritischer Persistenzpfad darf von LocalStorage abhängig sein.

3.2.10 N-MINUS-EINS-WOHNMECHANIK

Implementiere die verborgene Systemregel:

- reguläre verfügbare Wohn-/Schlafplätze bleiben unter der Zahl aktiver relevanter Charaktere,
- bei Spielerzuwachs entsteht nicht automatisch vollständige Deckung,
- Spiel erklärt die Formel nicht direkt,
- alternative Schlafwege über Beziehungen, Bezahlung, Arbeit, Improvisation oder Außenübernachtung,
- Konsequenzen wirken kausal weiter,
- Anti-Softlock: Wohnungslosigkeit ist kein Game Over.

Der verborgene unbegrenzte Schlafraum muss als echtes Geheimnis vorhanden sein, aber ohne Questmarker.

3.2.11 GEHEIMNISSYSTEM

Implementiere:

- subtile Hinweise,
- wiederkehrende Motive,
- optionale Entdeckung,
- keine "Geheimnisse 3/10"-Anzeige,
- alternative Hinweiswege,
- nachträgliche Fairness: Lösung muss im Rückblick nachvollziehbar sein,
- essenzieller Hauptfortschritt darf nicht ausschließlich an einen extrem obskuren Pixelklick gebunden sein.

3.2.12 PERSÖNLICHKEITSFRAKTALE

Entscheidungen beeinflussen mehrere Systeme:

- Beziehungen,
- Skills,
- Ruf,
- Zugangsmöglichkeiten,
- Biografie,
- Gerüchte,
- wirtschaftliche Folgen,
- spätere Ereigniswahrscheinlichkeiten.

Vermeide einfache Moralachsen. Konsequenzen sollen neue Wege öffnen und schließen, nicht nur "gut/schlecht" markieren.

3.2.13 BEZIEHUNGSSYSTEM

Implementiere:

- asymmetrische Beziehungen,
- mehrere Dimensionen statt Einzelwert,
- Vertrauen,
- Nähe,
- Respekt,
- Schuld/Verpflichtung,
- Konflikt,
- optional Geheimwissen.

Beziehungen verändern konkrete Möglichkeiten wie Schlafplatz, Hilfe, Preise, Kooperation, Informationen und Eventbeteiligung.

3.2.14 RÄUME UND BUNKERINNENRÄUME

Implementiere:

- Wechsel aus Hauptansicht ins Bunkerinnere,
- initial beschädigte/leere Räume,
- Zustände wie unzugänglich, vermüllt, geräumt, improvisiert, funktional, ausgebaut,
- mehrere Entwicklungswege,
- Raumfunktionen,
- Raum-Biografie.

Kein Mikromanagementzwang: Schutt/Maler/Strom sind Entscheidungsknoten, keine endlosen Klickarbeiten.

3.2.15 BAU-/AUSBAUSYSTEM

Mindestens:

- Eigenarbeit,
- Netzwerk/Freunde,
- bezahlt/professionell,
- improvisiert/szenetypisch,
- eventbasierte Finanzierung/Mithilfe.

Unterschiedliche Wege dürfen dasselbe funktionale Ziel erreichen, müssen aber unterschiedliche Geschichte, Kosten, Risiken und Szeneeffekte erzeugen.

3.2.16 WIRTSCHAFT

Implementiere eine konsistente Buchlogik für:

- Charaktergeld,
- Gemeinschaftskasse,
- Einnahmen,
- Ausgaben,
- Löhne/Gagen,
- Beschaffung,
- Eventumsatz,
- Verlust/Diebstahl,
- Schulden.

Kontostände müssen aus Buchungen herleitbar sein. Keine stille Geldmutation.

3.2.17 BESCHAFFUNG

Ressourcen und Dienstleistungen über mehrere Wege:

- kaufen,
- leihen,
- tauschen,
- Beziehungen,
- Arbeit,
- improvisierte Beschaffung,
- riskante/zweifelhafte spielinterne Wege nur als fiktionale Systementscheidung ohne reale Anleitung zu Straftaten.

Mechanik bewertet Konsequenzen, nicht reale Durchführungsmethoden.

3.2.18 SOUNDSYSTEM

Implementiere:

- Komponenten,
- Zustand,
- Leistungs-/Qualitätswerte,
- Besitz,
- Aufbauort,
- Reparatur,
- Upgrade,
- Eventwirkung,
- sichtbare Entwicklung.

Keine übertriebene technische Simulation, wenn sie den Kernloop nicht verbessert.

3.2.19 LOKALE MP3-FUNKTION

Nutzer kann eine kleine definierte Zahl lokaler Audiodateien auswählen.

Implementiere:

- Dateiauswahl,
- Typ-/Größenvalidierung,
- lokale Wiedergabe,
- Lautstärke,
- Stop/Pause,
- sichere Fehlerbehandlung,
- keine automatische Übertragung,
- Visualisierung/Lichtreaktion optional und performant,
- Reduced-Motion-/Flash-Safe-Schalter.

Die Anwendung muss auch ohne eigene MP3 vollständig spielbar bleiben.

3.2.20 LICHT-/AUDIOVISUELLE REAKTION

Reaktive Atmosphäre:

- Beat-/Amplitude-abhängige Effekte soweit zuverlässig,
- fallback bei nicht analysierbarer Datei,
- Leistungsgrenzen,
- Flash-Safe,
- Reduced Motion.

Keine gesundheitlich riskante Standard-Stroboskopfrequenz.

3.2.21 GRAFFITI-WAND

Implementiere:

- Pieces/Tags,
- Autorenbezug,
- Bereiche/Sektoren,
- lokale Änderungen,
- Crossen,
- Tages-/Periodenzustände,
- Historie,
- finalisierte Zustände.

Graffiti kann Hinweise überdecken, aber historische Zustände oder andere legitime Informationswege dürfen den Hinweis reproduzierbar erhalten.

3.2.22 GRAFFITI-KONFLIKTAUFLÖSUNG

Nicht "letzter Upload gewinnt".

Definiere deterministische Regeln aus:

- Eventreihenfolge,
- gültigem Zeitraum,
- Ressourcen/Aufwand,
- regelbasiertem Status,
- zulässigen Cross-Aktionen.

Die Regel muss testbar, wiederholbar und gegen absichtliche triviale Dominanz abgesichert sein.

3.2.23 BUNKERRAT

Implementiere:

- gleiches Vorschlagsrecht,
- Antrag,
- Abstimmungszeitraum,

- anonyme Stimme,
- Teilnahmezählung,
- Ergebnis,
- Historie,
- Monats-/Periodenthema.

Szenestatus oder Geld dürfen die formale Stimme nicht stärker machen.

3.2.24 SYNCHRONISIERTE ABSTIMMUNG

Stimmen lokal erfassen und als validierbare Events synchronisieren.

Anonymität gegenüber anderen Spielern wahren, ohne doppelte Stimmen zu erlauben. Definiere ein für den Prototyp angemessenes lokales Identitäts-/Gerätevertrauensmodell.

3.2.25 GERÜCHTESYSTEM

Gerüchte besitzen:

- Ursprung,
- beteiligte Figuren,
- Inhalt/Claim ID,
- Glaubwürdigkeit,
- Verbreitungsweg,
- Mutation,
- Reichweite,
- Lebensdauer,
- mögliche Widerlegung.

Gerücht und objektives Ereignis strikt trennen.

3.2.26 EVENT-/PARTYPLANUNG

Implementiere:

- Termin,
- Ort,
- Kapazität,
- Acts,
- Preis,
- Kosten,
- Erreichbarkeit,
- Strom,
- Getränke,
- Werbung/Gerüchte,
- Personal,
- Risiko,
- erwartete Besuchersegmente.

Keine reine Zahlentabelle: Werte müssen mit Beziehungen, Ruf, Gerüchten und Infrastruktur gekoppelt sein.

3.2.27 ACTS/DJS/MUSIKER

Acts können:

- bezahlt,
- über Beziehungen gewonnen,
- getauscht/kooperativ eingebunden,
- absagen,
- Konflikte haben,

- andere Acts beeinflussen.

Keine einzelne beste Beschaffungsstrategie.

3.2.28 BESUCHERSIMULATION

Besucheranzahl und Verhalten kausal aus mindestens:

- Ruf,
- Acts,
- Preis,
- Zugang,
- Gerüchte,
- Wetter falls simuliert,
- Szenezugehörigkeit,
- vorherige Erfahrungen,
- Kapazität,
- Konkurrenzereignissen.

Zufall nur über reproduzierbaren Seed/definierte Randomquelle, damit Tests möglich bleiben.

3.2.29 EVENTPHASEN

Mindestens:

- Vorbereitung,
- Anreise,
- Opening,
- Peak,
- Late,
- Abbau/Nachwirkungen.

Ereignisse dürfen phasenabhängig auftreten.

3.2.30 EVENT-NACHWIRKUNG

Nach Event:

- Kasse,
- Schulden,
- Schäden,
- Ruf,
- Beziehungen,
- Gerüchte,
- Raumzustand,
- Bio,
- neue Kontakte,
- offene Verpflichtungen.

After-Action-Datensatz erzeugen.

3.2.31 STORYINTRO

Kurzes, schnell lesbares mystisches Intro:

- Wald,
- Bunker,
- nicht eindeutig identifiziertes Signal,
- subtil statt Infodump,
- Ei/Ursprungsszene,
- Puls/Wasser-/Signal-Motiv,

- Schale bricht,
- kleine Figur mit Rock/Rucksack/Kassettenrekorder.

Spieldauer des Intros kurz halten und überspringbar machen, ohne die wesentliche Ausgangsinformation zu verlieren.

3.2.32 SPILERENTSTEHUNG

Kein klassischer Charaktereditor als alleiniger Start.

Frühe Handlungen formen:

- erste visuelle Tendenzen,
- Skills,
- Beziehungen,
- Persönlichkeitsmerkmale,
- erste Bio.

Kosmetische Anpassung darf später möglich sein, aber biografische Spuren sind nicht beliebig wegzuschalten.

3.2.33 VERBORGENE META-STORY

Die Welt deutet zunächst Bunker-/Signalsuche an. Später werden Identität, Erinnerung, Entscheidung, Biografie und menschliche Selbstdeutung thematisch sichtbar.

Nicht früh erklären. Keine einzelne Textwand soll das Konzept verraten.

3.2.34 DREI WAHRHEITSEBENEN

Implementiere getrennt:

1. objektiver Ereignisbestand,
2. subjektives Wissen/Erinnerung einzelner Figuren,
3. offizielle/soziale Darstellung wie Rat, Gerüchte, Plakate.

UI und Bio müssen kenntlich machen können, aus welcher Ebene eine Aussage stammt.

3.2.35 CONTENT REGISTRY

Implementiere:

- Inbox,
- Validator,
- Quarantine,
- Registry,
- Lockfile,
- Aktivierung,
- Deaktivierung,
- Rollback,
- Kompatibilitätsprüfung.

Aktive Contentversionen müssen eindeutig reproduzierbar sein.

3.2.36 HOT-SWAP-KLASSEN

Definiere:

- sofort sicher austauschbar,
- nur bei Neustart,
- nur mit Migration,
- nicht während laufender Welt austauschbar.

Textwechsel soll typischerweise in der sichersten Klasse liegen.

3.2.37 PLUGIN-/ERWEITERUNGSMODELL

Plugins erhalten nur definierte Capabilities.

Kein Plugin darf:

- beliebig Corezustände überschreiben,
- unvalidierte Events schreiben,
- Registry umgehen,
- fremde Persistenzstores mutieren.

Für den Prototyp darf das Pluginmodell zunächst eingeschränkt sein, muss aber Architekturgrenzen real implementieren.

3.2.38 IMPORT/EXPORT

Implementiere ein versioniertes Paketformat mit:

- Manifest,
- Daten,
- Versionen,
- Checksums,
- optional Signaturinfo,
- Contentlock,
- Exportzeitpunkt.

Import zuerst Dry Run:

- parse,
- schema,
- checksum,
- compatibility,
- migration plan,
- conflict detection,
- preview,
- dann atomare Übernahme.

3.2.39 BACKUP/RESTORE

Implementiere:

- manuelles Backup,
- versioniertes Format,
- Restore-Dry-Run,
- Restore in frische Umgebung,
- verständliche Fehler.

Backup muss unabhängig von Browsercache exportierbar sein.

3.2.40 SYNCHRONISATION

Für den Offline-first-Prototyp:

- Events zwischen Geräten/Instanzen austauschbar machen,
- Duplikate erkennen,
- ungültige Events quarantänisieren,
- deterministische Zusammenführung,
- Autoren-/Geräteketten prüfen,
- Konflikte als explizite Resolution dokumentieren.

3.2.41 KEINE NAIVE GLOBALE HASHKETTE

Offline parallel entstandene gültige Aktionen dürfen nicht nur deshalb kollidieren, weil es keinen einzelnen globalen "letzten Hash" gibt.

Nutze getrennte Autoren-/Geräteketten und ein deterministisches kanonisches Merge-/Projektionsverfahren.

3.2.42 SYNC-KONVERGENZ

Teste mehrere Permutationen derselben gültigen Eventmenge:

- A dann B,
- B dann A,
- mehrfacher Import,
- teilweise bereits bekannte Daten.

Endzustand muss, soweit Regeln kommutativ/auflösbar definiert sind, identisch sein.

3.2.43 KONFLIKTE

Konfliktklassen explizit definieren:

- problemlos mergebar,
- regelbasiert auflösbar,
- erfordert Resolution Event,
- ungültig/quarantänisieren.

Keine stille Datenverwerfung.

3.2.44 MIGRATIONEN

Für jede persistente Schemaänderung:

- fromVersion,
- toVersion,
- preconditions,
- transformation,
- validation,
- rollback/backup requirement,
- test fixtures alt->neu.

Mehrstufige Migrationen müssen deterministisch sequenziert werden.

3.2.45 RECOVERY

Implementiere einen sicheren Recoverybereich:

- Diagnose lesen,
- Daten exportieren,
- Backup prüfen,
- letzten gültigen Snapshot erkennen,
- defekte Imports quarantänisieren,
- keine destructive Aktion ohne klare Bestätigung.

Recovery startet read-only, solange keine Reparaturaktion ausdrücklich gewählt wurde.

3.2.46 DIAGNOSTIK

Health Screen zeigt laienverständlich:

- App-Version,
- World-Version,
- Datenbankstatus,
- Contentstatus,

- letzter erfolgreicher Check,
- Speicherfähigkeit,
- Syncstatus,
- bekannte Quarantäneelemente,
- Exportmöglichkeit.

Technische Detailansicht separat.

3.2.47 LAIENFEHLERMELDUNGEN

Jeder relevante Fehler erhält:

- kurze Erklärung,
- "Was ist passiert?",
- "Sind meine Daten sicher?",
- "Was kann ich jetzt tun?",
- technischen Fehlercode optional aufklappbar.

Keine rohen Stacktraces im normalen UI.

3.2.48 AUTOMATISIERTE REGRESSION

Erzeuge einen einzigen Hauptcheck, der mindestens ausführt:

- Strukturprüfung,
- Format/Lint,
- Syntax,
- Schema,
- Unit,
- Contracts,
- Architektur,
- Integration,
- Migration,
- Replay,
- Sync,
- wichtige End-to-End-Smokes,
- Placeholder Scan,
- Dokumentations-/Manifestkonsistenz.

Der Befehl endet non-zero bei Pflichtfehlern.

3.2.49 GOLDEN FIXTURES

Halte kleine stabile Welten für Regression:

- frische Welt,
- Welt nach erstem Wohnkonflikt,
- Welt nach Graffiti-Cross,
- Welt nach Abstimmung,
- Welt nach Event,
- Welt mit Gerücht,
- Welt mit Contentupdate,
- Welt nach Migration,
- Welt mit Syncmerge.

Expected State/Fingerprint versionieren.

3.2.50 PROPERTY-/INVARIANT-TESTS

Mindestens:

- Geld entsteht nicht ohne Buchung,
- eine Stimme pro zulässiger Identität und Abstimmung,
- Event IDs bleiben eindeutig,
- Replay ist deterministisch,
- ungültige Events verändern keine Projektion,
- Importfehler verändert aktive Welt nicht,
- abgeschlossene historische Tageswand wird nicht rückwirkend überschrieben,
- CANON/DRAFT-Regeln werden eingehalten.

3.2.51 END-TO-END-KERNFLOW

Automatisiere oder reproduziere mindestens folgenden vollständigen Flow:

1. frische Welt starten,

2. Charakter initialisieren,

3. erste Entscheidung,

4. Schlafplatzproblem,

5. Beziehungshandlung,

6. Raum betreten/entwickeln,

7. Graffitiaktion,

8. Ratsantrag und Stimme,

9. Event planen,

10. Event durchführen,

11. Konsequenzen verbuchen,

12. Bio prüfen,

13. exportieren,
14. in frischer Instanz importieren,
15. identischen Kernzustand bestätigen.

3.2.52 MULTI-INSTANCE-TEST

Simuliere mehrere unabhängige Instanzen/Browserprofile:

- getrennt handeln,
- exportieren/synchronisieren,
- zusammenführen,
- Ergebnis vergleichen.

Mindestens drei Teilnehmer im Test; Architektur auf maximal fünf menschliche Kernspieler auslegen, sofern Master-Spezifikation nichts Neues festlegt.

3.2.53 LANGZEITSIMULATION

Erzeuge automatisierte oder kontrollierte simulierte lange Spielverläufe:

- viele Events,
- viele Beziehungen,
- viele Buchungen,
- Contentwechsel,
- Graffitihistorie,
- Biografie,
- Synczyklen.

Prüfe Speicher, Laufzeit, Replay und Datenwachstum.

3.2.54 ANTI-SOFTLOCK

Teste Situationen:

- kein Geld,
- kein Schlafplatz,
- schlechte Beziehungen,
- Eventverlust,
- abgesagter Act,
- beschädigter Raum,
- fehlende eigene MP3,
- Quarantäne eines Contentpakets.

Spiel muss weiter handlungsfähig bleiben oder einen klaren Recovery-/Alternativweg bieten.

3.2.55 CONTENTTESTS

Jedes Contentpaket:

- Schema,
- IDs,
- Referenzen,
- Lokalisierung,
- Textlänge,
- verbotene Duplikate,
- Canonstatus,
- Dependencies,
- Checksum.

Verwaiste Referenzen blockieren Aktivierung.

3.2.56 STORY-/HINWEISTEST

Geheimnisse prüfen auf:

- subtile Vorankündigung,
- mehrere Hinweisformen,
- rückblickende Plausibilität,
- keine versehentliche UI-Markierung,
- keine Sackgasse.

Hints dürfen nicht in technische Debugtexte ausweichen.

3.2.57 AUDIO-NEGATIVTESTS

Teste:

- ungültige Datei,
- zu große Datei,
- nicht unterstützter Codec,
- Nutzer bricht Auswahl ab,
- Datei verschwindet,
- AudioContext blockiert,
- Reduced Motion,
- kein Audio.

App bleibt stabil.

3.2.58 BROWSER-/PLATTFORMTEST

Mindestens:

- Chromium-basierter Browser unter Linux,
- Firefox unter Linux oder zweite Engine,
- eine weitere Desktopplattform, soweit Work-Umgebung/Testzugang dies ermöglicht.

Nicht verfügbare reale Plattformtests dürfen nicht als durchgeführt behauptet werden; stattdessen klare Restprüfung markieren und soweit möglich Standards-/Engineprüfung durchführen.

3.2.59 DOKUMENTATION WÄHREND IMPLEMENTIERUNG

Nicht am Ende aus dem Gedächtnis schreiben. Bei jeder Iteration:

- CHANGELOG,
- ADR falls nötig,
- Schema-/API-Doku,
- Change Record,
- Evidence,
- Migrationhinweis,
- Benutzerrelevante Änderung

aktualisieren.

3.2.60 BONUSFEATURE - BUNKER BLACKBOX / DETERMINISTISCHER SZENARIO-RECORDER

ZIEL

Implementiere ein integriertes, standardmäßig nicht störendes Diagnose- und Replay-Werkzeug, das fachliche Abläufe reproduzierbar macht und dadurch Regression, Support und Releaseprüfung deutlich verbessert.

FUNKTIONEN

A. Recorder

- startet nur bewusst über Diagnose-/Entwicklermodus,
- zeichnet fachliche Commands statt Mauskoordinaten auf,
- speichert verwendeten deterministischen Seed,
- speichert Versions- und Content-Lock-Fingerprints,
- protokolliert resultierende Event-IDs und definierte Checkpoints,
- markiert nicht deterministische externe Einflüsse ausdrücklich.

B. Replay

- kann ein Szenario in einer frischen isolierten Testwelt abspielen,
- darf dabei keine produktive Welt verändern,
- vergleicht erwartete und tatsächliche Checkpoints,
- vergleicht finalen World-/Projection-Fingerprint,
- meldet den ersten divergierenden Schritt,
- zeigt erwarteten und tatsächlichen Zustand strukturiert an.

C. Golden Scenarios

Mindestens folgende Szenarien als versionierte Golden-Replays mitliefern:

- 1. Fresh Start und erste Charakterprägung.**
- 2. N-minus-eins-Wohnkonflikt mit alternativer Schlaflösung.**
- 3. Graffiti setzen und Cross-Konflikt.**
- 4. Ratsantrag mit anonymer Abstimmung.**
- 5. Eventplanung, Durchführung und Nachwirkung.**
- 6. Export/Import und identischer Kernfingerprint.**
- 7. Zwei beziehungsweise mehr Instanzen mit Syncmerge.**
- 8. Contentupdate mit erfolgreicher Registry-Aktivierung.**
- 9. Abgelehnter fehlerhafter Contentimport mit unveränderter Welt.**
- 10. Recovery aus definiertem beschädigtem Testzustand.**

D. Bug Capsule

Ein exportierbares Supportpaket enthält nur notwendige Diagnoseinformationen:

- bugCapsuleVersion,
- scenario oder minimierte Commandfolge,
- anonymisierte technische IDs soweit möglich,
- Core-/Schema-/Ruleset-/Content-Versionen,
- World Fingerprint vor/nach Reproduktion,
- relevante Fehlercodes,
- strukturierte Logs für die betroffene Correlation-ID,
- optional reproduzierbare Fixturedaten.

Explizit ausgeschlossen:

- lokale MP3-Dateiinhalte,

- lokale Nutzerbilder,
- Passwörter/Secrets,
- unnötige Freitexte,
- fremde personenbezogene Informationen.

E. Replay Diff

Für jede Divergenz:

- Schritindex,
- Command ID,
- Expected Fingerprint,
- Actual Fingerprint,
- betroffene Projection/Entity,
- minimaler strukturierter Zustandsdiff,
- Correlation ID,
- Fehlercode.

Große Vollzustände nicht unkontrolliert in UI oder Logs kippen.

F. RELEASE COCKPIT

Im Diagnosebereich eine kompakte Ansicht:

- Build/Release-ID,
- Health-Status,
- letzter vollständiger Regressionlauf,
- Golden-Replay-Status,
- letzter Clean-Room-Status,
- Quarantäneanzahl,
- Backupstatus,
- Export Bug Capsule,
- technischer Evidence-Link/Pfad soweit in der Plattform möglich.

Die normale Spieleroberfläche darf dadurch nicht überladen werden.

SICHERHEIT UND ROBUSTHEIT

- Replay immer isoliert vom aktiven Spielstand.
- Recorder und Replay verwenden dieselben öffentlichen Application-/Domain-Schnittstellen wie reguläre Aktionen.
- Keine Sonderlogik, die nur im Testpfad funktioniert.
- Szenarien versionieren und bei inkompatiblen Schemaänderungen migrieren oder eindeutig ablehnen.
- Bug Capsules mit Schema validieren und mit Checksum versehen.
- Ein fehlerhaftes Replay darf niemals die aktive Welt mutieren.

ABNAHME

Die BUNKER BLACKBOX gilt nur als fertig, wenn:

1. ein aufgezeichnetes fachliches Szenario exportiert werden kann,
2. dasselbe Szenario in einer frischen isolierten Welt reproduzierbar abläuft,

3. Expected und Actual Fingerprint bei unverändertem Code identisch sind,

4. eine absichtlich veränderte Erwartung zuverlässig als Divergenz erkannt wird,
5. die erste Abweichungsstelle verständlich angezeigt wird,
6. eine Bug Capsule erzeugt und wieder eingelesen werden kann,

7. Privacy-Tests bestätigen, dass MP3-/Bildinhalte nicht enthalten sind,

8. mindestens zehn Golden-Replays grün laufen.

3.2.61 STUFE-2-ERGEBNIS

Stufe 2 endet erst, wenn:

- sämtliche Pflichtsysteme implementiert,
- Kernflows verbunden,
- automatisierte Prüfungen grün,
- keine releaseblockierenden TODOs/Platzhalter,
- keine bekannten Datenverlustpfade,
- keine roten Architekturverletzungen,
- ein vollständiger Release Candidate erzeugbar ist,
- die BUNKER BLACKBOX mindestens zehn Golden-Replays reproduzierbar grün ausführt,
- und mindestens eine Bug Capsule erfolgreich round-trip-validiert wurde.

Danach ohne Unterbrechung mit Stufe 3 fortfahren.

STUFE 3 - RELEASEHÄRTUNG, ABSCHLUSSPRÜFUNG UND DOKUMENTATION

ZIEL

Nicht nur "funktioniert bei mir", sondern reproduzierbar releasefähig.

3.3.1 RELEASE CANDIDATE FREEZE

Erzeuge RC-Version.

Ab diesem Punkt:

- nur Bugfixes,
- Dokumentationskorrekturen,
- Releaseblocker,
- keine neuen Features.

Jede RC-Änderung startet betroffene Regression neu.

3.3.2 CLEAN-ROOM-TEST

Kopiere/entpacke den RC in ein frisches Verzeichnis bzw. neue Prüfworkspace.

Keine versteckten Entwicklungsdateien, Caches oder globale lokale Zustände verwenden.

Starte ausschließlich nach dokumentierter Anleitung.

3.3.3 FRISCHINSTALLATION

Prüfe:

- Voraussetzungen,
- Startkommando,
- Browseraufruf,
- erste Welt,
- persistenter Reload,
- Offlineverhalten.

Dokumentation muss exakt dem realen Ablauf entsprechen.

3.3.4 VOLLSTÄNDIGE AUTOMATISCHE SUITE

Alle Pflichtchecks erneut ausführen und Evidence speichern:

- Format,
- Lint,
- Syntax,
- Schema,
- Unit,
- Contract,
- Architektur,
- Integration,
- Migration,
- Replay,
- Sync,
- E2E,
- Security negative,
- Accessibility smoke,
- Performance smoke,

- Placeholder scan,
- Broken-link/reference scan,
- package/manifest validation,
- Golden-Replay-Suite der BUNKER BLACKBOX,
- Bug-Capsule-Schema-/Privacy-/Round-Trip-Test.

3.3.5 MANUELLE KRITISCHE SMOKES

Mindestens manuell:

- Erststart,
- Bunker-Hauptansicht,
- Charakterkarte,
- Bio,
- Wohnungssituation,
- Geheimraum-Hinweisweg,
- Innenraum,
- Graffiti,
- Rat,
- Event,
- Audio mit und ohne Datei,
- Export,
- Import,
- Recovery,
- BUNKER BLACKBOX Recorder,
- Replay eines Golden Scenarios,
- verständliche Divergenzanzeige mit absichtlich verändertem Testcheckpoint,
- Export einer datensparsamen Bug Capsule.

Ergebnisse protokollieren.

3.3.6 DATENINTEGRITÄT

Prüfe:

- Eventreplay,
- Hashes,
- Signaturen soweit aktiv,
- beschädigtes Event,
- doppeltes Event,
- Reihenfolgevarianten,
- Snapshot vs Replay,
- Export/Import-Fingerprint.

3.3.7 REGRESSION NACH LETZTEM FIX

Der absolut letzte Codefix erfordert mindestens:

- betroffene Unit-/Integrationtests,
- vollständigen Hauptcheck,
- Kern-E2E,
- Clean-Room-Smoke.

Der Release darf nicht direkt nach einem ungetesteten "kleinen letzten Fix" erstellt werden.

3.3.8 PERFORMANCE-ABSCHLUSS

Messe definierte Budgets mit repräsentativer Welt.

Dokumentiere:

- Messgerät/Browser,
- Datensatzgröße,
- Ergebnis,
- Grenzwert,
- PASS/FAIL.

Keine erfundenen Zahlen.

3.3.9 ACCESSIBILITY-ABSCHLUSS

Prüfe:

- Tastaturpfad,
- Fokus,
- Labels,
- Kontrast,
- Zoom,
- Reduced Motion,
- Flash Safe,
- Fehlermeldungen.

Blockierende Bedienprobleme vor Release beheben.

3.3.10 SECURITY-/ROBUSTNESS-ABSCHLUSS

Prüfe:

- gefährliche Imports,
- manipulierte JSONs,
- ungültige Contentpakete,
- DOM-Injection-Versuche,
- überschrittene Dateigrenzen,
- beschädigte Backups,
- unerwartete Versionen.

System muss ablehnen/quarantänisieren statt aktive Welt zu beschädigen.

3.3.11 KNOWN ISSUES

Erstelle ehrliche Liste:

- keine releaseblockierenden Fehler,
- nicht blockierende Einschränkungen mit Workaround,
- nicht getestete Umgebungen,
- bewusst verschobene Post-Prototype-Funktionen.

Nicht als "keine bekannten Fehler" deklarieren, wenn Prüfungen Lücken zeigen.

3.3.12 RELEASE MANIFEST

Enthält:

- Produktname,
- Releaseversion,
- Build-/Commit-/Fingerprint,
- Datum,
- Core-Version,
- Schema-Version,
- World-Format,
- Content Lock,
- getestete Browser/Plattformen,

- Checksums der Releaseartefakte.

3.3.13 RELEASEPAKET

Liefern:

- lauffähiges Projekt,
- notwendige Assets,
- keine Entwicklungsreste,
- Startskript/Kommando,
- Checks,
- Dokumentation,
- Lizenz-/Third-Party-Hinweise soweit vorhanden,
- Changelog,
- Release Notes,
- Evidence Report.

ZIP zusätzlich nach Erstellung wieder in frischem Ziel entpacken und erneut Smoke/Hash prüfen.

3.3.14 ENTWICKLERDOKUMENTATION - PFLICHTUMFANG

Erstelle professionelle technische Dokumentation mit mindestens:

A. System Overview

- Ziel und Scope,
- Architekturdiagramm in Text/Mermaid sofern geeignet,
- Module,
- Datenfluss,
- Eventfluss.

B. Repository Guide

- vollständiger Verzeichnisbaum,
- Verantwortlichkeiten,
- wo neuer Content hingehört,
- wo neue Regeln hingehören,
- wo niemals direkt geschrieben werden darf.

C. Local Development

- Voraussetzungen,
- Installation,
- Start,
- Tests,
- Build/Package,
- Debugging.

D. Domain Model

- Characters,
- Relationships,
- Rooms,
- Housing,
- Graffiti,
- Council,
- Economy,

- Rumors,
- Events,
- Biography.

E. Event/Ledger

- Eventstruktur,
- IDs,
- Hash/Signatur,
- Replay,
- Projection,
- Correction.

F. Persistence

- IndexedDB-Schema,
- Transaktionen,
- Snapshots,
- Migrationen,
- Backup.

G. Sync

- Export/Import,
- Device/Author Chain,
- Merge,
- Conflict Resolver,
- Quarantine.

H. Content

- Registry,
- Manifest,
- Canon,
- Version,
- Hot-Swap,
- neue Textdatei hinzufügen.

I. Plugins

- Capabilities,
- Grenzen,
- Validierung,
- Beispiel für sichere Erweiterung.

J. Testing

- Testpyramide,
- Fixturestrategie,
- Regression,
- E2E,
- Performance,
- Accessibility.

K. Error Handling

- Fehlercodes,
- Recoverability,
- Diagnose,
- Supportexport.

L. Release

- Versionierung,
- Freeze,
- RC,
- Checklist,
- Fingerprint,
- Rollback.

M. Maintenance

- Text ändern,
- Contentpaket austauschen,
- Schema erweitern,
- Migration anlegen,
- neue Regel hinzufügen,
- neue UI-Komponente hinzufügen,
- Regression prüfen.

3.3.15 DREISTUFIGE LAIENANLEITUNG - PFLICHT

Die finale Benutzer-/Betreiberanleitung muss zusätzlich in drei klar getrennten Schwierigkeitsstufen vorliegen:

STUFE A - SCHNELLSTART FÜR LAIEN

Ziel: Spiel in wenigen Minuten öffnen und sicher ausprobieren.

Enthält ausschließlich:

- 1. Was brauche ich?**
- 2. Paket entpacken.**
- 3. Startbefehl oder Startdatei.**
- 4. Browser öffnen.**
- 5. Neues Spiel starten.**
- 6. Wo werden Daten gespeichert?**
- 7. Wie beende ich sicher?**
- 8. Wie mache ich sofort ein Backup?**

Mit exakten Kommandos, keine Variablen-Platzhalter.

STUFE B - NORMALER BETRIEB

Ziel: Nutzer kann das Projekt selbst pflegen.

Enthält:

- 1. Start/Stop.**
- 2. Spielstand exportieren.**
- 3. Spielstand importieren.**
- 4. Backup erstellen.**
- 5. Backup prüfen.**
- 6. Content-/Textupdate installieren.**
- 7. Diagnose öffnen.**
- 8. Fehlerbericht exportieren.**
- 9. Update durchführen.**

10. sichere Rückkehr zur letzten funktionierenden Version.

Jeder Schritt:

- Klick-/Kommandoanweisung,

- erwartetes Ergebnis,
- Fehlerfall,
- sichere Abhilfe.

STUFE C - WARTUNG OHNE PROGRAMMIERKENNTNISSE

Ziel: fortgeschrittener Laie kann typische Projektpflege ausführen, ohne Core-Code zu verändern.

Enthält:

1. Textpakete ändern,

2. neue Contentdatei nach Vorlage anlegen,

3. Manifestversion korrekt erhöhen,

4. Validator ausführen,

5. Quarantäne verstehen,

6. Regressioncheck ausführen,

7. Evidence lesen,

8. ZIP erzeugen,

9. frischen Prüflauf durchführen,

10. Rollback.

Explizit zeigen, welche Dateien ein Laie ändern darf und welche nicht.

3.3.16 TROUBLESHOOTING

Mindestens tabellarisch:

- Symptom,
- wahrscheinliche Ursache,
- Datenrisiko,
- Diagnose,
- Lösung,
- wann Recovery benutzen.

Fälle:

- Seite bleibt leer,
- DB kann nicht geöffnet werden,
- Import wird abgelehnt,
- Content in Quarantäne,
- Audio spielt nicht,
- Speicherplatzproblem,
- Synckonflikt,
- falsche Version,
- beschädigtes Backup,
- Test rot.

3.3.17 RELEASE-EVIDENCE-REPORT

Erstelle einen finalen Bericht:

- RC/Release ID,
- Prüfdatum,
- Umgebung,
- ausgeführte Befehle,
- alle Gates,
- PASS/FAIL,
- Testzahlen,
- bekannte Einschränkungen,
- Artefaktchecksums,
- finaler Fingerprint,
- Aussage, ob Releasekriterien erfüllt sind.

Nur tatsächliche Ergebnisse eintragen.

3.3.18 ABSOLUT LETZTER TECHNISCHER SCHRITT

Nach Fertigstellung ALLER Dateien und Dokumente:

1. finalen Releaseordner erzeugen,

2. Release-ZIP erzeugen,

3. SHA-256 berechnen,

4. ZIP in ein vollständig neues Prüfverzeichnis entpacken,

5. dort die dokumentierte Laien-Schnellstart-Anleitung verwenden,

6. Anwendung starten,

7. vollständigen Release-Hauptcheck ausführen,

8. Kern-E2E ausführen,

9. Export/Import-Smoke durchführen,

10. vollständige BUNKER-BLACKBOX-Golden-Replay-Suite ausführen,
11. eine Bug Capsule aus dem frischen Release erzeugen und in einer zweiten isolierten Testwelt reproduzieren,
12. erzeugten Fingerprint mit Release Manifest vergleichen,

13. Dokumentlinks und Dateipfade prüfen,

14. Evidence Report final aktualisieren,

15. erneut SHA-256 der finalen Artefakte berechnen, falls der Evidence Report Bestandteil des Pakets ist,
16. keine Datei mehr ändern, nachdem die endgültigen Checksums festgeschrieben wurden.

DIES IST DER VERBINDLICHE LETZTE IMPLEMENTIERUNGS- UND PRÜFSCHRITT.

Der Auftrag darf nicht vorher als abgeschlossen bezeichnet werden.

4. QUALITÄTSGATES - RELEASE NUR BEI GRÜN

G0 SCOPE GATE

Alle Prototyp-Pflichtanforderungen besitzen implementierten Status oder ausdrücklich genehmigten Non-Scope-Status. Keine stillen Auslassungen.

G1 STRUCTURE GATE

Repo, Manifest, Architekturgrenzen, State Owner, Versionsdateien konsistent.

G2 STATIC GATE

Format, Lint, Syntax, Placeholder Scan grün.

G3 SCHEMA/CONTENT GATE

Alle aktiven strukturierten Daten und Contentpakete validieren.

G4 UNIT/CONTRACT GATE

Fachregeln, Invarianten, Modulverträge grün.

G5 DATA GATE

Persistence, Migration, Replay, Import/Export, Recovery grün.

G6 SYNC/REPLAY GATE

Duplikate, Reihenfolgevarianten, Merge, Konflikte, Quarantäne sowie deterministische Golden-Replays grün.

G7 UX/ROBUSTNESS GATE

Kernflows, Accessibility Smokes, Fehlerzustände, Anti-Softlock grün.

G8 RELEASE GATE

Clean Room, Fresh Start, Kern-E2E, BUNKER-BLACKBOX-Golden-Replays, Bug-Capsule-Round-Trip, Package, Checksums, Dokumentation und Evidence grün.

REGEL

Ein rotes Pflichtgate blockiert Release. Gelb ist nur zulässig, wenn es sich ausdrücklich um einen nicht releaseblockierenden dokumentierten Umgebungs-/Kompatibilitätshinweis handelt und kein Daten-/Kernfunktionsrisiko besteht.

5. DEFINITION OF DONE FÜR JEDE ITERATION

Eine Iteration ist nur DONE, wenn:

- 1. Scope umgesetzt.**
- 2. Kein unbeabsichtigter Scope Creep.**
- 3. Tests hinzugefügt/angepasst.**
- 4. Bestehende relevante Tests grün.**
- 5. Fehlerpfade berücksichtigt.**
- 6. Logging/Diagnose soweit relevant vorhanden.**
- 7. Schemas/Manifeste aktualisiert.**
- 8. Dokumentation aktualisiert.**
- 9. Change Record erstellt.**
- 10. Regression Evidence gespeichert.**
- 11. Rollbackweg bekannt.**
- 12. Keine neuen offenen technischen Platzhalter.**
- 13. Betroffene Architekturgrenzen eingehalten.**
- 14. Versionierung korrekt.**
- 15. Nutzerrelevante Änderungen im Changelog.**
- 16. Fachreview durchgeführt.**
- 17. Iteration eingefroren oder bei späterem Eingriff sauber REOPEN dokumentiert.**
- 18. Wenn Domain-/Workflowverhalten betroffen ist:
bestehendes Golden-Replay weiterhin grün oder
neues/angepasstes Szenario mit dokumentierter Begründung
vorhanden.**

6. INTELLIGENTES FEHLERHANDLING

6.1 FEHLERKLASSEN

- INFO: kein Eingriff nötig.
- WARNING: Betrieb möglich, Beobachtung nötig.
- RECOVERABLE_ERROR: Aktion fehlgeschlagen, Welt unverändert/sicher.
- QUARANTINE_ERROR: externe/importierte Daten isoliert.
- DATA_RISK: möglicher Datenverlust/Inkonsistenz; Schreibbetrieb stoppen.
- FATAL_BOOT: App kann Kernzustand nicht sicher laden; Recovery anbieten.

6.2 TRANSAKTIONSPRINZIP

Bei mehrstufigen Mutationen:

- vorvalidieren,
- atomar schreiben,
- nachvalidieren,
- bei Fehler kein halbgeschriebener sichtbarer Zustand.

6.3 IMPORTPRINZIP

Import ist read/validate/plan/apply, niemals direkt apply.

6.4 RETRY

Nur idempotente oder sicher deduplizierbare Operationen automatisch wiederholen. Keine blinden Retries bei fachlichen Mutationen.

6.5 FAIL CLOSED VS FAIL OPEN

Bei Integrität, Import, Schema, Rechtegrenzen: fail closed.

Bei rein kosmetischen optionalen Features: graceful degradation.

6.6 DATENRETTUNG

Auch bei rotem Health Status muss Export valider Roh-/Backupdaten soweit sicher möglich bleiben.

7. REGRESSIONSDISZIPLIN

7.1 JEDER BUG BRAUCHT NACH MÖGLICHKEIT EINEN TEST

Vor Fix: reproduzierender Test rot.

Nach Fix: Test grün.

Falls technisch nicht automatisierbar: dokumentierter manueller Regressionfall.

7.2 KEIN "KLEINER FIX" OHNE REGRESSION

Auch Einzeiler können Datenpfade brechen.

7.3 GOLDEN WORLDS

Stabile Fixture-Welten mit Fingerprints.

7.4 REPLAY REGRESSION

Alte gültige Welten müssen nach kompatiblen Updates korrekt replaysen oder migrieren.

7.5 CONTENT REGRESSION

Alte Contentpakete:

- kompatibel weiter funktionieren,
- sauber migriert werden,
- oder verständlich und sicher abgelehnt werden.

7.6 PERFORMANCE REGRESSION

Signifikante Verschlechterungen in Replay, Start oder großen Biografien blockieren Release, wenn Budgets verletzt werden.

7.7 BUNKER-BLACKBOX-REGRESSION

Für jeden reproduzierbaren komplexen Bug soll nach Möglichkeit eine minimierte Bug Capsule oder ein dauerhaftes Golden Scenario entstehen. Dadurch wird ein einmal gefundener Fehler zu einem automatischen Schutz gegen sein Wiederauftreten.

7.8 SZENARIO-MINIMIERUNG

Bei langen Fehlerreproduktionen nach Möglichkeit die kleinste Commandfolge ermitteln, die den Fehler noch zuverlässig erzeugt. Das minimierte Szenario erhält eigene ID, Ursache, Fix-Version und Expected Fingerprint.

8. WARTBARKEIT UND VEREINHEITLICHUNG

8.1 EIN WEG PRO AUFGABE

Eine zentrale Funktion für:

- Geld buchen,
- Beziehung ändern,
- Event schreiben,
- Content laden,
- Fehler erzeugen,
- ID erzeugen,
- Version vergleichen,
- Import validieren.

Keine parallelen Schattenimplementierungen.

8.2 REGISTRY STATT HARDCODING

Content, Eventtypen, Fehlertypen, Screens, Capabilities und migrationsrelevante Typen soweit sinnvoll zentral registrieren.

8.3 KLEINE DATEIEN MIT EINER VERANTWORTUNG

Große "god files" aufteilen, aber nicht künstlich hunderte Einzeiler erzeugen.

8.4 ÖFFENTLICHE SCHNITTSTELLEN KLEIN HALTEN

Module exportieren nur benötigte APIs.

8.5 DEPRECATION STATT SOFORTIGEM BRUCH

IDs/APIs/Contentreferenzen mit kontrolliertem Migrationspfad ändern.

8.6 DOKUMENTATION ALS TEIL DES SYSTEMS

Doku wird in Regression auf:

- vorhandene Pfade,
- gültige Kommandos,
- Versionsnummern,
- Links/Referenzen

geprüft.

9. PRODUKTSPEZIFISCHE NICHT VERHANDELBARE SPIEL-DNA

9.1

Das Spiel bestraft Entscheidungen nicht nur; es erinnert sich an sie und lässt Konsequenzen weiterwirken.

9.2

Die Welt zeigt Möglichkeiten, ohne jede davon als Quest zu erklären.

9.3

Charaktere werden durch Erlebtes geprägt; Charaktererstellung ist teilweise Spielhandlung.

9.4

Biografie entsteht aus tatsächlichen Ereignissen.

9.5

Formale Gleichheit verhindert keine entstehende soziale Hierarchie.

9.6

Knappheit erzeugt Beziehungen und Geschichten, aber keine unlösbaren Sackgassen.

9.7

Ein scheinbarer Nachteil kann exklusiven Zugang zu anderer Information oder Story eröffnen.

9.8

Räume und Objekte besitzen Geschichte, nicht nur Level.

9.9

Mehrere Wege können dasselbe Ziel erreichen, aber nicht dieselbe Geschichte erzeugen.

9.10

Gerücht, Erinnerung, offizieller Beschluss und objektives Ereignis dürfen auseinanderfallen.

9.11

Geheimnisse bleiben Geheimnisse: keine Komplettierungsanzeige.

9.12

Der Bunker wird durch Spielgeschichte sichtbar individuell.

9.13

Graffiti ist sozialer Konflikt, Kunst, Historie und möglicher Hinweisraum.

9.14

Der Rat organisiert gemeinsame Absicht, aber Beschluss ersetzt nicht Umsetzung.

9.15

Musik, Soundsystem und Veranstaltungen bilden den zentralen kulturellen Spielkern.

9.16

Mystery/Existenzthema bleibt subtil und wird nicht durch Infodumps entwertet.

10. VERBINDLICHER RELEASE-SCOPE DES PROTOTYPEN

Der Prototyp muss mindestens folgende Dinge real erlebbar machen:

1. Start/Intro.

- 2. funktionierende Hauptansicht.
- 3. persistente Charaktere.
- 4. die drei festen Storyfiguren.

5. Charakterkarte.

- 6. frühe spielerische Charakterprägung.

7. Skills/Ruf.

8. Beziehungen.

- 9. n-minus-eins-Wohnen.
- 10. mehrere Schlafalternativen.
- 11. verborgener Schlafraum mit subtiler Hinweislogik.

12. Bunkerinnenraum.

- 13. mindestens mehrere ausbaubare Räume/Bereiche.
- 14. mehrere Ausbauewege.

15. Raumhistorie.

- 16. konsistente Wirtschaft.

17. Gemeinschaftskasse.

18. Soundsystementwicklung.

- 19. lokale MP3-Wiedergabe.
- 20. sichere Audio-/Lichtreaktion oder sauberer Fallback.

21. Graffitiwand.

22. Crossmechanik.

- 23. historische Wandzustände.

24. Bunkerrat.

25. anonyme Abstimmung.

26. Monats-/Periodenthema.

27. Gerüchte.

28. Eventplanung.

29. Acts.

30. Besucherberechnung.

31. Eventphasen.

32. Missgeschicke/Nachwirkungen.

33. Bio.

34. drei Wahrheitsebenen.

35. Content Registry.

36. Contentimport/Quarantäne.

37. Event Ledger.

38. Replay.

39. IndexedDB.

40. Export/Import.

41. Backup/Restore.

42. Offline-Sync zwischen mindestens simulierten Instanzen.

43. deterministische Zusammenführung.

44. Recovery Screen.

45. Health Screen.

- 46. automatisierte Regression.
- 47. laiengerechter Schnellstart.

48. Entwicklerdokumentation.

- 49. releasefähiges ZIP/Paket.
- 50. finaler Clean-Room-Abschlusstest.

51. BUNKER BLACKBOX mit deterministischem Recorder, Replay, Golden Scenarios, Diff und datensparsamer Bug Capsule.

Wenn ein Scopepunkt bewusst nur minimal für den Prototyp umgesetzt wird, muss er dennoch vollständig funktionieren und darf nicht als Attrappe vorhanden sein.

11. AUSGABE- UND ARTEFAKTSTRUKTUR AM ENDE

Erzeuge mindestens:

/release/

- SOUND_SYSTEM-IKER_<version>.zip
- SHA256SUMS.txt
- RELEASE_MANIFEST.json
- RELEASE_NOTES.txt
- CHANGELOG.md

/docs/

- 01_SCHNELLSTART_LAIEN.pdf oder .md/.txt
- 02_BETRIEB_UND_BACKUP.pdf oder .md/.txt
- 03_WARTUNG_FUER_LAIEN.pdf oder .md/.txt
- ENTWICKLERDOKUMENTATION.md
- ARCHITEKTUR.md
- DATENMODELL.md
- CONTENT_GUIDE.md
- SYNC_RECOVERY.md
- TESTING_RELEASE.md
- TROUBLESHOOTING.md
- BUNKER_BLACKBOX_REPLAY.md

/evidence/

- FINAL_RELEASE_EVIDENCE.txt
- test-results/
- fingerprints/
- compatibility/
- manual-smoke/
- golden-replays/
- bug-capsules/

Die konkreten Formate dürfen an das Repository angepasst werden. Inhalt darf nicht fehlen.

12. ABSCHLUSSANTWORT VON CHATGPT WORK

Erst nachdem der absolut letzte Clean-Room-Schritt grün abgeschlossen ist, darf die Abschlussantwort formuliert werden.

Sie muss knapp, aber vollständig enthalten:

1. Releaseversion.

2. Was tatsächlich implementiert wurde.

3. Status aller Gates G0-G8.

4. Anzahl/Art ausgeführter Tests.

- 5. getestete Umgebungen.
- 6. bekannte nicht blockierende Einschränkungen.
- 7. finaler Projekt-/Releasefingerprint.

8. SHA-256 der Releaseartefakte.

9. Pfade/Links zu:

- Releasepaket,
- Entwicklerdokumentation,
- dreistufiger Laienanleitung,
- Evidence Report.

10. Bestätigung, dass der finale ZIP-Clean-Room-Lauf nach der letzten Änderung durchgeführt wurde.

11. Ergebnis der BUNKER-BLACKBOX-Golden-Replay-Suite und Pfad zu mindestens einer validierten Bug Capsule.

VERBOTEN IN DER ABSCHLUSSANTWORT:

- "müsste noch",
- "sollte später",
- "kann anschließend implementiert werden"

für einen verpflichtenden Release-Scopepunkt.

Wenn ein Pflichtgate nicht grün ist, darf nicht "fertig" oder "releasefähig" behauptet werden. Dann muss weitergearbeitet werden, sofern der Blocker innerhalb der verfügbaren Projektumgebung lösbar ist.

13. BONUSFEATURE-VERTRAG - BUNKER BLACKBOX

13.1 PRODUKTNUTZEN

Die BUNKER BLACKBOX verbindet Entwicklung, Fehlersuche und Laienbetrieb. Ein Nutzer kann einen problematischen Ablauf als Supportpaket exportieren; ein Entwickler oder ChatGPT Work kann denselben Ablauf deterministisch in einer isolierten Welt reproduzieren.

13.2 KEIN CHEAT-/SPOILERWERKZEUG IM NORMALEN SPIEL

Das Cockpit ist Diagnosefunktion. Es darf verborgene Geheimnisse, versteckte Regeln oder interne Storyflags im normalen Spielmodus nicht offenlegen. Erweiterte technische Daten nur im bewusst geöffneten Diagnose-/Entwicklermodus.

13.3 DETERMINISMUSGRENZE

Externe Einflüsse wie echte lokale Uhrzeit, nicht deterministische Browserereignisse oder Dateiverfügbarkeit müssen:

- entweder durch abstrahierte Inputs/Seeds kontrolliert,
- oder als nicht reproduzierbare Abhängigkeit im Szenario markiert werden.

Kein Replay darf falsche Determinismusgarantien behaupten.

13.4 PRIVACY BY DEFAULT

Bug Capsules arbeiten nach Datensparsamkeit:

- Allowlist statt blindem Vollbackup,
- lokale Medien standardmäßig ausgeschlossen,
- Freitext nach Möglichkeit referenzieren/minimieren,
- technische Gerätekennungen pseudonymisieren,
- Exportvorschau vor manueller Weitergabe.

13.5 GOLDEN-REPLAY-POLITIK

Golden Scenarios sind Releaseartefakte. Änderungen an Regeln, die erwartete Fingerprints verändern, benötigen:

- fachliche Begründung,
- Change Record,
- aktualisierte Erwartung,
- Regressionnachweis,
- und Review durch Domain sowie QA.

13.6 BUG-TO-TEST-PIPELINE

Ein bestätigter reproduzierbarer Bug folgt idealerweise:

Bug Capsule -> minimiertes Szenario -> roter Regressionstest -> Fix -> grünes Szenario -> Aufnahme in Regression, wenn der Fehler kritisch oder wiederholungsgefährdet ist.

13.7 LAIENNUTZEN

In der dreistufigen Laienanleitung erklären:

- wie man im Fehlerfall eine Bug Capsule erzeugt,
- welche Daten sie enthält,
- welche ausdrücklich nicht enthalten sind,
- wie sie lokal gespeichert wird,
- und wie man sie wieder einliest, ohne den echten Spielstand zu überschreiben.

13.8 RELEASEBEDINGUNG

Ein Release ohne funktionsfähige BUNKER BLACKBOX darf nur dann erfolgen, wenn diese Masteranweisung ausdrücklich durch eine neuere freigegebene Version ersetzt wurde. In v1.1 ist sie verpflichtender Prototyp-Scope.

14. STARTANWEISUNG

Beginne jetzt mit STUFE 1.

Behandle diese Datei als obersten operativen Arbeitsvertrag. Lade zuerst die vorhandenen Projektartefakte und Masterdokumente, ermittle ihren tatsächlichen Stand und arbeite anschließend autonom durch STUFE 2 und STUFE 3.

Nutze die vorhandenen Projektartefakte als Quelle und bewahre kompatible, bereits grüne Implementierungen. Ersetze funktionierenden Code nicht aus Stilgründen. Ändere nur, wenn dadurch eine Anforderung, ein Fehler, eine Wartbarkeitsverletzung, ein Sicherheitsproblem oder ein nachgewiesenes Qualitätsdefizit behoben wird.

Halte den Nutzer während längerer Arbeit in sinnvollen Abständen mit kurzen Statusmeldungen auf dem Laufenden. Keine niedrigen Detailprotokolle im Chat; technische Details gehören in Change Records und Evidence.

Der Auftrag endet ausschließlich nach dem finalen, frischen, reproduzierbaren Clean-Room-Abschlusslauf und der Erstellung aller geforderten Dokumente und Releaseartefakte.